

GPIO#	2nd func.	Pin#	Pin#	2nd func.	GPIO#
	+3.3 V	1	2	+5 V	
2	SDA1 (I ² C)	3	4	+5 V	
3	SCL1 (I ² C)	5	6	GND	
4	GCLK	7	8	TXD0 (UART)	14
	GND	9	10	RXD0 (UART)	15
17	GEN0	11	12	GEN1	18
27	GEN2	13	14	GND	
22	GEN3	15	16	GEN4	23
	+3.3 V	17	18	GEN5	24
10	MOSI (SPI)	19	20	GND	
9	MISO (SPI)	21	22	GEN6	25
11	SCLK (SPI)	23	24	CE0_N (SPI)	8
	GND	25	26	CE1_N (SPI)	7
(Pi 1 Models A and B stop here)					
EEPROM	ID_SD	27	28	ID_SC	EEPROM
5	N/A	29	30	GND	
6	N/A	31	32		12
13	N/A	33	34	GND	
19	N/A	35	36	N/A	16
26	N/A	37	38	Digital IN	20
	GND	39	40	Digital OUT	21

Figure 1: Raspberry Pi pinout diagram

use only *GPIO.BOARD* here.

Step 3: Set up the necessary input and output pins.

Step 4: Read inputs and give outputs.

(Steps 3 and 4 are extensively explained in the following sections.)

Step 5: Clean up GPIO and exit.

You must clean up the pin set-ups before your program exits otherwise those pin settings will persist, and that might cause trouble when you use the same pins in another program. The Pi 'expresses its displeasure' with a warning. To clean up the entire set of pins, invoke *GPIO.cleanup()*. If you want only a few pins to be cleaned up, then the pin numbers should be provided as *GPIO.cleanup(channel_list)*. Anyway, you can suppress the warning messages by calling *GPIO.setwarnings(False)*.

Setting up an output pin

To set a pin as an output pin, we have to call the method *GPIO.setup(pin, GPIO.OUT)*. Here, *pin* is the pin number of the pin. You can set the initial state to be *ON(1)* or *OFF(0)* by using the *initial* parameter. So, to set Pin 8 as an output pin with the initial state as *OFF*, use the following code:

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BOARD)
```

```
mypin = 8
GPIO.setup(mypin, GPIO.OUT, initial = 0)
```

Now, to change the state of the pin, type:

```
GPIO.output(pin, state)
```

The value of *state* can be either *1* or *GPIO.HIGH* or *True* for the ON state and *0* or *GPIO.LOW* or *False* for OFF.

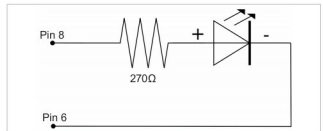


Figure 2: Circuit 1

Practice circuit 1

For all the practice circuits, use a breadboard and jumper wires.

Now, take a regular 5mm LED, connect the anode (long leg) to any GPIO pin, say Pin 8, using a series resistor of 270Ω. Connect the cathode (short leg) to any ground pin, say Pin 6.

The series resistor makes sure that the LED will not draw too much current. The GPIO pins provide 3.3V, with the maximum permitted current draw of 50mA. Roughly speaking, for 3v3, any resistor less than or equal to 1K can be used as the series resistor, and lower the resistance to the appropriate value if the LED is too dim. The exact value can be calculated using the formula:

$$\text{Resistor value} = (3.3 - V_{\text{LED}}) / I_{\text{LED}}$$

For more information, you can visit <http://www.electricaltechnology.org/2013/08/how-to-calculate-value-of-resistor-for-LED-circuits.html>

After the connections have been made properly, power up the Pi and open IDLE or any text editor. Then type the following code:

```
import RPi.GPIO as GPIO
#for the sleep method
import time
led = 8
#set numbering mode for the program
GPIO.setmode(GPIO.BOARD)
#setup led(pin 8) as output pin
GPIO.setup(led, GPIO.OUT, initial=0)
try:
    #turn on and off the led in intervals of 1 second
    while(True):
        #turn on, set as HIGH or 1
```